

Unit 4

Express js and Angular js

Exam-Ready Study Notes

Unit 04 :- Express js and Angular js

■ Definition

- Express.js is a fast, minimal, and flexible web application framework for Node.js that simplifies the process of building web servers, APIs, and backend applications.

■ Introduction

In Node.js, developers can create web servers using the built-in http module, but it requires:

- Writing more code
- Handling routing manually
- Managing request/response logic explicitly

To overcome these complexities, Express.js was introduced.

■ Express provides:

- A simplified way to create servers
- Built-in support for routing and middleware
- Easy handling of HTTP requests and responses

It is widely used in:

- REST API development
- Backend of web applications
- MERN stack (MongoDB, Express, React, Node.js)

■ Why Express.js is Needed

Before Express:

```
const http = require("http");

http.createServer((req, res) => {
  res.write("Hello World");
  res.end();
}).listen(3000);
```

■ Problems:

- No routing system
- Hard to scale
- More boilerplate code

With Express:

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Hello World");
});

app.listen(3000);
```

■ Benefits:

- Clean and readable
- Easy routing
- Less code

■ Features of Express.js (Detailed)

1. Minimal and Lightweight Framework
 - Express does not enforce strict structure
 - Developers have full control

■ Makes it flexible for small and large applications

2. Fast Execution (Non-blocking I/O)
 - Built on Node.js event-driven architecture
 - Handles multiple requests simultaneously

■ Suitable for:

- Real-time apps
 - High-traffic applications
3. Powerful Routing System
 - Allows handling different HTTP methods:

* GET

* POST

* PUT

* DELETE

■ Example:

```
app.get("/users", (req, res) => {});
app.post("/users", (req, res) => {});
```

4. Middleware Support (Very Important)

Middleware functions execute between request and response cycle.

■ Uses:

- Logging
- Authentication
- Data parsing

■ Example:

```
app.use((req, res, next) => {  
  console.log("Request received");  
  next();  
});
```

5. Request and Response Handling

- Express simplifies:
- Request processing (`req`)
- Sending responses (`res`)

■ Example:

```
res.send("Hello");  
res.json({ name: "John" });
```

6. Integration with Databases

- Easily connects with:
- MongoDB
- MySQL

7. REST API Development

- Express is widely used to create APIs

■ Example:

```
app.get("/api/students", (req, res) => {  
  res.json([{ name: "John" }]);  
});
```

■ Steps to Set Up Express Application**■ Step 1: Install Node.js**

- Required to run Express

■ Step 2: Initialize Project

```
npm init -y
```

■ Step 3: Install Express

```
npm install express
```

■ Step 4: Create Server File

■ File: `app.js`

■ Example (Detailed Express Server)

```
const express = require("express");

// Create app
const app = express();

// Middleware
app.use(express.json());

// Route 1
app.get("/", (req, res) => {
  res.send("Welcome to Express Server");
});

// Route 2
app.get("/students", (req, res) => {
  res.json([
    { name: "John", age: 22 },
    { name: "Mike", age: 21 }
  ]);
});

// Route 3
app.post("/students", (req, res) => {
  const data = req.body;
  res.send("Student Added: " + JSON.stringify(data));
});

// Start server
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

■ Explanation of the Above Code

- `express()` → Creates application instance
- `app.use()` → Middleware usage
- `app.get()` → Handles GET request
- `app.post()` → Handles POST request
- `req.body` → Gets request data
- `res.send()` → Sends response
- `res.json()` → Sends JSON response
- `app.listen()` → Starts server

■ Real-World Use Case

■ Example: E-commerce Backend

- `/products` → Get all products
- `/orders` → Place order
- `/users` → Manage users

Express handles:

- Routing
- Data processing
- API responses

■ Advantages of Express.js

1. Simplifies Backend Development
2. Reduces Code Complexity
3. High Performance
4. Scalable for Large Applications
5. Strong Ecosystem

■ Disadvantages of Express.js

1. No Strict Structure
 - Developers must organize code manually
2. Middleware Complexity
 - Hard for beginners
3. Callback/Async Complexity

■ Conclusion

Express.js is a powerful and flexible framework that simplifies Node.js development by:

- Providing routing
- Supporting middleware
- Handling HTTP requests efficiently

It plays a key role in full stack development, especially in building APIs and backend services.

TIP: Exam Writing Tips

- Start with definition (Express = Node.js framework)
- Explain why Express is needed (compare with HTTP module)
- Write features (minimum 5 points)
- Include middleware concept (very important)
- Provide one detailed server example
- Use keywords: *routing, middleware, REST API, request/response*

Configuring Routes in Express.js

■ Definition

- Routing in Express.js refers to the process of defining how an application responds to different client requests (URLs and HTTP methods).

■ Introduction

In a web application, users interact through different URLs such as:

- `/home`
- `/login`
- `/students`

Each URL should return a specific response. This is handled using routing.

■ In Express:

- Routes are defined using methods like:
- `app.get()`
- `app.post()`
- `app.put()`
- `app.delete()`

Routing is a core concept in Express and is heavily used in:

- Web applications
- REST APIs

■ Basic Structure of a Route

```
app.METHOD(PATH, HANDLER)
```

■ Where:

- `METHOD` → HTTP method (GET, POST, etc.)
- `PATH` → URL path
- `HANDLER` → Function to handle request

■ Types of Routing Methods

1. GET Method

Used to retrieve data from server

```
app.get("/", (req, res) => {  
  res.send("Home Page");  
});
```

2. POST Method

Used to send data to server

```
app.post("/login", (req, res) => {  
  res.send("Login Successful");  
});
```

3. PUT Method

Used to update existing data

```
app.put("/user", (req, res) => {  
  res.send("User Updated");  
});
```

4. DELETE Method

Used to delete data

```
app.delete("/user", (req, res) => {  
  res.send("User Deleted");  
});
```

■ Route Parameters (Dynamic Routing)

Used to pass values in URL.

```
app.get("/students/:id", (req, res) => {  
  res.send("Student ID: " + req.params.id);  
});
```

■ Example:

`/students/101` → Output: Student ID: 101

■ Query Parameters

Used to send additional data in URL.

```
app.get("/search", (req, res) => {  
  res.send("Query: " + req.query.name);  
});
```

■ Example:

`/search?name=John`

■ Multiple Routes Example


```
const express = require("express");
const app = express();

// Home Route
app.get("/", (req, res) => {
  res.send("Welcome");
});

// About Route
app.get("/about", (req, res) => {
  res.send("About Page");
});

// Contact Route
app.get("/contact", (req, res) => {
  res.send("Contact Page");
});

app.listen(3000);
```

■ Route Handling (Advanced)

1. Multiple Handlers

```
app.get("/example",
  (req, res, next) => {
    console.log("First handler");
    next();
  },
  (req, res) => {
    res.send("Second handler");
  }
);
```

2. Router Module (Modular Routing)

Used to organize routes in separate files.

```
const express = require("express");
const router = express.Router();

router.get("/students", (req, res) => {
  res.send("Student List");
});

module.exports = router;
```

■ In main file:

```
app.use("/", require("./routes"));
```

■ Real-World Example

■ College Management System

Route	Purpose
-----	-----
<code>/students`</code>	Get all students
<code>/students/:id`</code>	Get student by ID
<code>/students/add`</code>	Add new student
<code>/students/delete`</code>	Delete student

■ Advantages of Routing

1. Organized Code
 - Separate logic for each URL
2. Easy Maintenance
 - Easy to update routes
3. Supports REST APIs
 - Structured API development
4. Flexible
 - Dynamic routing supported

■ Disadvantages

1. Complex for Large Apps (without structure)
2. Requires Proper Planning
3. Too Many Routes Can Reduce Readability

■ Conclusion

Routing is a fundamental concept in Express.js that determines how the server responds to client requests. It enables:

- URL handling
- API creation
- Dynamic data processing

Mastering routing is essential for building scalable backend applications.

TIP: Exam Writing Tips

- Start with definition of routing
- Write basic structure (`app.METHOD(PATH, HANDLER)`)
- Explain GET, POST, PUT, DELETE
- Include route parameters and query parameters
- Add one complete example
- Mention Router module (extra marks)

- Use keywords: *routing, URL, HTTP methods, parameters*

Using Request Objects (`req`) in Express.js

■ Definition

The Request Object (`req`) in Express.js represents the HTTP request sent by the client and contains all the information about the request such as parameters, query strings, headers, and body data.

■ Introduction

Whenever a client (browser or application) sends a request to the server, Express automatically creates a request object (`req`).

■ This object helps developers to:

- Access user input
- Read data sent by client
- Handle dynamic requests

It is one of the most important objects in Express.

■ Structure of Request-Response Cycle

```
Client → Request (req) → Server → Response (res) → Client
```

■ `req` contains incoming data

■ `res` sends output

■ Common Properties of Request Object

1. `req.params` (Route Parameters)

Used to access dynamic values in URL

```
app.get("/user/:id", (req, res) => {  
  res.send("User ID: " + req.params.id);  
});
```

■ Example: `/user/101`

■ Output: User ID: 101

2. `req.query` (Query Parameters)

Used to access data sent in URL query string

```
app.get("/search", (req, res) => {  
  res.send("Search: " + req.query.name);  
});
```

■ Example: `/search?name=John``

■ Output: Search: John

3. `req.body`` (Request Body)

Used to access data sent in request body (POST/PUT)

```
app.use(express.json());  
  
app.post("/user", (req, res) => {  
  res.send("Name: " + req.body.name);  
});
```

■ Requires middleware: `express.json()``

4. `req.headers``

Contains HTTP headers

```
app.get("/", (req, res) => {  
  console.log(req.headers);  
  res.send("Headers Logged");  
});
```

5. `req.method``

Returns HTTP method

```
app.get("/", (req, res) => {  
  res.send(req.method);  
});
```

■ Output: GET

6. `req.url``

Returns requested URL

```
app.get("/about", (req, res) => {  
  res.send(req.url);  
});
```

■ Output: /about

■ Example (Complete Request Handling)

```
const express = require("express");
const app = express();

app.use(express.json());

app.post("/student/:id", (req, res) => {
  const id = req.params.id;
  const name = req.body.name;
  const course = req.query.course;

  res.send(
    "ID: " + id +
    ", Name: " + name +
    ", Course: " + course
  );
});

app.listen(3000);
```

■ Example URL:

`POST /student/101?course=React`

■ Body:

```
{ "name": "John" }
```

■ Output:

`ID: 101, Name: John, Course: React`

■ Real-World Example

■ Login System

- Username & password → `req.body`
- User ID → `req.params`
- Filters/search → `req.query`

■ Advantages of Request Object

1. Easy Data Access
 - Access all request data in one object
2. Supports Multiple Data Types
 - URL, body, headers
3. Essential for Dynamic Applications
 - Handles user input

■ Disadvantages

1. Requires Middleware Setup (`req.body`)

2. Improper Use May Cause Errors
3. Security Risks if Not Validated

■ Conclusion

The Request Object (`req``) is a fundamental part of Express.js that allows developers to:

- Access client data
- Handle dynamic requests
- Build interactive applications

Understanding `req`` is essential for backend development.

TIP: Exam Writing Tips

- Start with definition of req object
- Explain req.params, req.query, req.body (very important)
- Include one combined example
- Mention express.json() middleware
- Use keywords: *request data, parameters, query, body*

Using Response Objects (`res``) in Express.js

■ Definition

The Response Object (`res``) in Express.js is used to send a response from the server to the client. It contains methods that allow the server to return data such as text, JSON, status codes, or files.

■ Introduction

After a client sends a request (`req``), the server must send back a response. This is done using the response object (`res``).

■ The `res`` object helps to:

- Send output to client
- Return data in different formats
- Set HTTP status codes
- End the request-response cycle

■ Without `res``, the client will not receive any response.

■ Request-Response Cycle

```
Client → Request (req) → Server → Response (res) → Client
```

■ `req`` → receives data

■ `res`` → sends data

■ Common Methods of Response Object

1. `res.send()`

- Sends response in text or HTML format

```
app.get("/", (req, res) => {  
  res.send("Hello World");  
});
```

2. `res.json()`

- Sends response in JSON format

```
app.get("/user", (req, res) => {  
  res.json({ name: "John", age: 22 });  
});
```

■ Automatically sets content-type to JSON

3. `res.status()`

- Sets HTTP status code

```
app.get("/error", (req, res) => {  
  res.status(404).send("Page Not Found");  
});
```

4. `res.sendFile()`

- Sends a file as response

```
res.sendFile(__dirname + "/index.html");
```

5. `res.redirect()`

- Redirects to another URL

```
res.redirect("/home");
```

6. `res.end()`

- Ends the response without sending data

```
res.end();
```

■ Example (Complete Response Handling)

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Welcome to Home Page");
});

app.get("/user", (req, res) => {
  res.json({ name: "Mike", age: 21 });
});

app.get("/error", (req, res) => {
  res.status(404).send("Page Not Found");
});

app.listen(3000);
```

■ Combining Request and Response

```
app.get("/student/:id", (req, res) => {
  const id = req.params.id;
  res.send("Student ID is " + id);
});
```

■ `req` → gets data

■ `res` → sends output

■ Real-World Example

■ Login API

```
app.post("/login", (req, res) => {
  if (req.body.password === "123") {
    res.status(200).json({ message: "Login Successful" });
  } else {
    res.status(401).json({ message: "Invalid Password" });
  }
});
```

■ Advantages of Response Object

1. Flexible Output
 - Send text, JSON, files
2. Easy API Development
 - Useful for REST APIs
3. Status Code Control
 - Helps in error handling

4. Simple Syntax

- Easy to use

■ Disadvantages

1. Must End Response Properly
2. Incorrect Use Can Crash App
3. Requires Understanding of HTTP Codes

■ Conclusion

The Response Object (`res`) is essential in Express.js for:

- Sending data back to the client
- Handling API responses
- Controlling output format and status

It completes the request-response cycle and plays a crucial role in backend development.

TIP: Exam Writing Tips

- Start with definition of res object
- Explain res.send(), res.json(), res.status() (very important)
- Include example with status code
- Combine req + res in one example
- Use keywords: *response, output, JSON, status code*

EXPRESS PART COMPLETED

You now have full coverage of:

1. Getting Started with Express
2. Routing
3. Request Object
4. Response Object

PART 2 :- ANGULAR

Importance of Angular

■ Definition

- Angular is a TypeScript-based front-end framework developed by Google used to build dynamic, single-page web applications (SPAs) with a structured and scalable approach.

■ Introduction

In modern web development, users expect:

- Fast loading
- Smooth navigation
- Interactive UI

Traditional web applications reload the entire page for every request, which leads to:

- Slow performance
- Poor user experience

To solve this problem, frameworks like Angular were introduced.

■ Angular helps in:

- Building Single Page Applications (SPA)
- Creating dynamic and responsive user interfaces
- Managing large-scale applications efficiently

■ Why Angular is Important

1. Builds Single Page Applications (SPA)
 - Loads a single HTML page
 - Updates content dynamically without reloading

■ Improves user experience

2. Component-Based Architecture
 - Application is divided into small reusable components

■ Example:

- Navbar
 - Footer
 - Login form
3. Two-Way Data Binding
 - Automatically syncs data between:
 - UI (View)
 - Logic (Component)

■ Reduces manual coding

4. Dependency Injection
 - Manages services efficiently
 - Promotes modular design
5. TypeScript Support
 - Angular uses TypeScript (superset of JavaScript)

■ Benefits:

- Strong typing
 - Better error detection
 - Improved code quality
6. Routing Support
 - Enables navigation between pages without reload
 7. Built-in Tools and Features
 - Forms handling
 - HTTP client
 - Testing tools

■ No need for external libraries

8. Suitable for Large Applications

- Structured architecture
- Easy to maintain

■ Real-World Applications of Angular

- Google applications
- E-commerce websites
- Admin dashboards
- Banking applications

■ Example:

- Online shopping website with dynamic product updates

■ Example (Simple Angular Concept)

HTML (View)

```
<h1>{{ message }}</h1>
```

Component (TypeScript)

```
export class AppComponent {  
  message = "Hello Angular";  
}
```

■ Output: Hello Angular

■ This shows data binding

■ Advantages of Angular

1. Structured Framework
 - Organized code
2. Reusability
 - Components can be reused
3. High Performance
 - Faster UI updates

Strong Community Support

- Backed by Google
- 5. Built-in Features
 - No need for many external libraries

■ Disadvantages of Angular

1. Steep Learning Curve
 - Difficult for beginners
2. Complex Syntax
 - Requires understanding of TypeScript
3. Heavy Framework
 - Larger bundle size

■ Conclusion

Angular is a powerful front-end framework that plays a crucial role in modern web development by:

- Creating dynamic user interfaces
- Supporting SPA architecture
- Providing structured and scalable solutions

It is widely used for building enterprise-level applications.

TIP: Exam Writing Tips

- Start with definition (Angular = TypeScript-based framework)
- Mention SPA concept (very important)
- Explain component-based architecture
- Include two-way data binding
- Add real-world use case
- Use keywords: *SPA, components, data binding, TypeScript*

Understanding Angular (Features + Architecture)

■ Definition

- Angular is a TypeScript-based front-end framework developed by Google that follows a component-based architecture to build dynamic and scalable web applications.

■ Introduction

Angular is not just a library—it is a complete framework that provides everything needed to build modern web applications, including:

- UI development
- Routing
- Data handling
- HTTP communication

■ It is mainly used for creating Single Page Applications (SPAs) where:

- Pages do not reload
- Content updates dynamically

■ Key Features of Angular

1. Component-Based Architecture
 - Application is divided into independent components
 - Each component handles:

- HTML (View)
- CSS (Style)
- TypeScript (Logic)

■ Example:

- Header component
 - Login component
2. Two-Way Data Binding
- Automatically synchronizes:
 - Model (data)
 - View (UI)

■ Reduces manual DOM manipulation

3. Dependency Injection (DI)
- Provides required services automatically
 - Promotes reusable and modular code
4. Routing
- Enables navigation between pages without reload

■ Example:

- `/home`
 - `/about`
5. Directives
- Special instructions in HTML
 - Modify DOM behavior

■ Examples:

- `\*ngIf` → Conditional display
 - `\*ngFor` → Loop
6. TypeScript Support
- Strongly typed language
 - Helps in:
 - Error detection
 - Better code quality
7. Built-in Services
- HTTP client
 - Form handling
 - Validation
8. Modular Structure
- Application is divided into modules
 - Improves organization

Angular Architecture — Main Components

1. Modules (NgModules)

■ Definition

An Angular Module (NgModule) is a container that groups related components, directives, pipes, and services into a single unit.

■ Explanation

- Every Angular application has a root module called `AppModule`
- Modules help in:
 - Organizing code
 - Improving scalability
 - Lazy loading

■ Types of Modules:

- Root Module
- Feature Module
- Shared Module

■ Example

```
import { NgModule } from '@angular/core';

@NgModule({
  declarations: [],
  imports: [],
  providers: [],
  bootstrap: []
})
export class AppModule {}
```

■ Key Points

- `declarations` → Components, directives
- `imports` → Other modules
- `providers` → Services
- `bootstrap` → Root component

2. Components

■ Definition

A Component is the basic building block of Angular UI that controls a part of the screen.

■ Explanation

Each component consists of:

- HTML (Template) → UI
- CSS (Styles) → Design

- TypeScript (Class) → Logic

■ Angular apps are made of multiple components.

■ Example

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: '<h1>Hello Angular</h1>'
})
export class AppComponent {}
```

■ Key Points

- Reusable
 - Modular
 - Controls specific UI part
3. Templates

■ Definition

A Template is the HTML view of a component that defines how data is displayed.

■ Explanation

- Uses Angular syntax:
- `{{ }}` → Interpolation
- `\*ngIf`, `\*ngFor` → Directives

■ Example

```
<h1>{{ name }}</h1>
```

■ Displays dynamic data from component

■ Key Points

- Defines UI
 - Supports data binding
 - Dynamic rendering
4. Services

■ Definition

A Service is a class used to share data and business logic across components.

■ Explanation

- Keeps logic separate from UI

- Used for:
- API calls
- Data sharing
- Business logic

■ Example

```
export class DataService {  
  getData() {  
    return ["Angular", "React"];  
  }  
}
```

■ Key Points

- Promotes code reusability
 - Centralized logic
 - Used with Dependency Injection
5. Dependency Injection (DI)

■ Definition

- Dependency Injection is a design pattern used to provide services to components automatically.

■ Explanation

- Angular injects required services into components
- No need to create objects manually

■ Example

```
constructor(private dataService: DataService) {}
```

■ Key Points

- Reduces code complexity
 - Improves maintainability
 - Promotes loose coupling
6. Directives

■ Definition

- Directives are special instructions in Angular that modify the behavior or appearance of DOM elements.

■ Types of Directives

■ (a) Structural Directives

- Change DOM structure

■ Examples:


```
<div *ngIf="isLoggedIn">Welcome</div>
<div *ngFor="let item of items">{{ item }}</div>
```

■ (b) Attribute Directives

- Change appearance

■ Example:

```
<p [style.color]='red'>Text</p>
```

■ Key Points

- Enhance HTML functionality
- Control UI behavior

7. Data Binding

■ Definition

- Data Binding is the process of connecting component data with the view (UI).

■ Types of Data Binding

1. Interpolation

```
{{ name }}
```

2. Property Binding

```
<img [src]="imageUrl">
```

3. Event Binding

```
<button (click)="clickMe()">Click</button>
```

4. Two-Way Binding

```
<input [(ngModel)]="name">
```

■ Key Points

- Synchronizes UI and data
- Reduces manual DOM updates

■ Architecture Summary (Very Important)

```
Module → Component → Template → Data Binding → View
      ↓
      Service (via Dependency Injection)
```

Angular architecture is built using:

- Modules → Organize application
- Components → Build UI
- Templates → Display data
- Services → Handle logic
- DI → Provide services
- Directives → Control DOM
- Data Binding → Connect UI and data

■ **This structured approach makes Angular suitable for large and scalable applications.**

■ **Angular Architecture Flow (Important)**

```
Component → Template → Data Binding → View → User Interaction → Component
```

■ **This cycle keeps UI updated dynamically**

■ **Example (Simple Angular Structure)**

Component

```
export class AppComponent {
  name = "John";
}
```

Template

```
<h1>Hello {{ name }}</h1>
```

■ **Output: Hello John**

■ **Advantages of Angular**

1. Complete Framework
2. Strong Structure
3. Reusability of Components
4. Built-in Features
5. Suitable for Large Applications

■ **Disadvantages**

1. Complex for Beginners

2. Heavy Framework
3. Requires TypeScript Knowledge

■ Conclusion

Angular is a powerful and structured framework that provides:

- Component-based development
- Built-in tools
- Efficient architecture

Its architecture makes it ideal for large-scale and enterprise applications.

TIP: Exam Writing Tips

- Start with definition (Angular framework)
- List features (minimum 5 points)
- Explain architecture components (modules, components, services)
- Include architecture flow diagram
- Add small example
- Use keywords: *component-based, DI, modules, data binding*

Creating a Basic Angular Application

■ Definition

Creating a basic Angular application refers to the process of installing Angular CLI, setting up a project, understanding its file structure, and running the application to build dynamic web applications.

■ Introduction

Angular applications are developed using a tool called Angular CLI (Command Line Interface), which simplifies:

- Project creation
- File management
- Running and building applications

■ Instead of manually writing configurations, Angular CLI automates everything, making development faster and structured.

■ Prerequisites

Before installing Angular, the following are required:

1. Install Node.js
- Download from:

■ [Node.js Official Website](https://nodejs.org?utm_source=chatgpt.com)

- Verify installation:

```
node -v  
npm -v
```

■ Installing Angular CLI (Very Important)

■ Step 1: Install Angular CLI Globally

```
npm install -g @angular/cli
```

■ This installs Angular CLI globally in your system

■ Step 2: Verify Installation

```
ng version
```

■ Displays Angular CLI version

■ Steps to Create Angular Application

■ Step 1: Create New Project

```
ng new myApp
```

■ During setup:

- It asks for routing → Yes/No
- Choose stylesheet format (CSS recommended)

■ Step 2: Navigate to Project Folder

```
cd myApp
```

■ Step 3: Run the Application

```
ng serve
```

■ Open browser:

- http://localhost:4200

■ Complete Angular Project Structure (Very Important)

After creating the project, Angular generates a structured folder system.

■ Root Level Files

```
myApp/  
■  
■■■ node_modules/  
■■■ src/  
■■■ angular.json  
■■■ package.json  
■■■ tsconfig.json
```

■ Explanation

1. `node\_modules`
 - Contains all installed dependencies
 - Managed by npm
2. `package.json`
 - Stores:
 - Project details
 - Dependencies
 - Scripts
3. `angular.json`
 - Configuration file
 - Controls build and project settings
4. `tsconfig.json`
 - TypeScript configuration

■ Inside `src` Folder

```
src/  
■  
■■■ app/  
■■■ assets/  
■■■ index.html  
■■■ main.ts  
■■■ styles.css
```

1. `index.html`
 - Main HTML file
 - Entry point of application
2. `main.ts`
 - Bootstrap file
 - Starts Angular application
3. `styles.css`
 - Global styles
4. `assets`
 - Stores images, files

■ Inside `app/` Folder (Core Part)

```
app/  
■  
■■■ app.component.ts  
■■■ app.component.html  
■■■ app.component.css  
■■■ app.module.ts
```

■ Detailed Explanation

1. `app.component.ts`
 - Contains logic of application

```
export class AppComponent {  
  title = "My Angular App";  
}
```

2. `app.component.html`
 - Defines UI (View)

```
<h1>{{ title }}</h1>
```

3. `app.component.css`
 - Styling for component
4. `app.module.ts` (Very Important)
 - Root module of application

```
import { NgModule } from '@angular/core';  
  
@NgModule({  
  declarations: [],  
  imports: [],  
  bootstrap: []  
})  
export class AppModule {}
```

■ Creating a Basic Component

■ Step 1: Generate Component

```
ng generate component hello
```

■ Step 2: Component Code

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello',
  templateUrl: './hello.component.html'
})
export class HelloComponent {
  message = "Hello Angular";
}
```

■ Step 3: Template Code

```
<h1>{{ message }}</h1>
```

■ Step 4: Use Component

```
<app-hello></app-hello>
```

■ Example (Simple Angular Application)

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: '<h1>Welcome to Angular</h1>'
})
export class AppComponent {}
```

■ Output: Welcome to Angular

■ Advantages of Angular Application Setup

1. Fast Development using CLI
2. Organized File Structure
3. Easy Component Creation
4. Built-in Development Server

■ Disadvantages

1. Initial Setup Complexity
2. Requires CLI Knowledge
3. Large Project Size

Advantages of Angular

1. Fast Development
Creates project instantly

2. Standard Structure
Organized file system
3. Built-in Server
Easy testing using ng serve
4. Code Generation
Auto-generates components

■ Disadvantages

1. Requires Learning CLI Commands
2. Initial Setup Time
3. Heavy for Small Pro

■ Conclusion

Creating an Angular application involves:

- Installing Node.js and Angular CLI
- Generating project using CLI
- Understanding folder structure
- Running application

Angular CLI provides a standardized and efficient way to build scalable applications.

TIP: Exam Writing Tips

- Start with definition of Angular application setup
- Clearly write Angular CLI installation (npm install -g @angular/cli)
- Include commands: ng new, ng serve
- Draw or write complete file structure (very important for marks)
- Explain app folder files (app.component, app.module)
- Include simple component example
- Use keywords: *Angular CLI, project structure, components*

Angular Components

■ Definition

An Angular Component is the basic building block of an Angular application that controls a part of the user interface (UI) and contains HTML (template), CSS (styles), and TypeScript (logic).

■ Introduction

Angular applications are built using a component-based architecture, where the entire application is divided into small, reusable parts called components.

■ Each component:

- Represents a portion of the UI
- Handles its own logic
- Can be reused across the application

■ Example:

- Header component
- Footer component
- Login form component

■ Structure of an Angular Component

Each component consists of three main parts:

1. TypeScript File (Logic)
 - Contains business logic
 - Handles data and functions
2. HTML Template (View)
 - Defines the UI
 - Displays data
3. CSS File (Style)
 - Provides styling

■ Component Syntax (Very Important)

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: '<h1>Hello Angular</h1>',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = "My App";
}
```

■ Explanation of Component Parts

1. `@Component` Decorator
 - Defines metadata for component

■ Contains:

- `selector` → HTML tag name
 - `template` → UI
 - `styleUrls` → CSS file
2. Selector

```
selector: 'app-root'
```

■ Used in HTML as:

```
<app-root></app-root>
```

3. Template

```
template: '<h1>Hello</h1>'
```

■ Defines UI

4. Class

```
export class AppComponent {  
  title = "Angular";  
}
```

■ Contains variables and functions

■ Creating a Component

■ Step 1: Generate Component

```
ng generate component student
```

■ Step 2: Files Created

```
student/  
■  
■■■ student.component.ts  
■■■ student.component.html  
■■■ student.component.css
```

■ Example (Complete Component)

■ student.component.ts

```
import { Component } from '@angular/core';  
  
@Component({  
  selector: 'app-student',  
  templateUrl: './student.component.html'  
})  
export class StudentComponent {  
  name = "John";  
  age = 22;  
}
```

■ student.component.html

```
<h2>Student Name: {{ name }}</h2>
<p>Age: {{ age }}</p>
```

■ Usage in app.component.html

```
<app-student></app-student>
```

■ Output:

- Student Name: John
- Age: 22

■ Types of Components

1. Root Component
 - Main component (`AppComponent`)
2. Child Components
 - Used inside other components

■ Advantages of Components

1. Reusability
 - Same component can be reused
2. Modularity
 - Divides application into parts
3. Maintainability
 - Easy to update
4. Separation of Concerns
 - UI and logic separated

■ Disadvantages

1. Too Many Components Can Increase Complexity
2. Requires Proper Structure
3. Learning Curve for Beginners

■ Conclusion

Angular components are the foundation of Angular applications. They:

- Control UI
- Manage data
- Enable modular development

Using components, developers can build scalable and maintainable applications efficiently.

TIP: Exam Writing Tips

- Start with definition (component = building block)
- Explain 3 parts: HTML, CSS, TypeScript

- Write @Component syntax (very important)
- Include example with selector usage
- Mention types (root & child)
- Use keywords: *component-based, selector, template*

Angular Expressions

■ Definition

- Angular Expressions are JavaScript-like expressions used within Angular templates to display dynamic data from the component into the view (HTML).

■ Introduction

In Angular applications, the UI is not static. It changes based on data. Angular expressions help in:

- Displaying dynamic values
- Performing simple operations
- Binding data from component to template

■ Angular expressions are written inside:

```
{{ expression }}
```

■ This is called Interpolation.

■ Syntax of Angular Expressions

```
{{ expression }}
```

■ Example:

```
<h1>{{ name }}</h1>
```

■ Displays value of `name` from component

■ Features of Angular Expressions

1. Similar to JavaScript
 - Supports variables and operators
2. Used in Templates Only
 - Written inside HTML files
3. Evaluated Dynamically
 - Automatically updates when data changes
4. Safe Execution

- Prevents errors (e.g., null checks)

■ Types of Angular Expressions

1. Displaying Variables

```
name = "John";
```

```
<h1>{{ name }}</h1>
```

■ Output: John

2. Performing Operations

```
<p>{{ 10 + 5 }}</p>
```

■ Output: 15

3. Using Object Properties

```
student = { name: "Mike", age: 22 };
```

```
<p>{{ student.name }}</p>
```

■ Output: Mike

4. Calling Functions

```
getMessage() {  
  return "Hello Angular";  
}
```

```
<p>{{ getMessage() }}</p>
```

■ Output: Hello Angular

5. Conditional Expressions

```
<p>{{ age > 18 ? "Adult" : "Minor" }}</p>
```

■ Example (Complete Angular Expression)

■ Component

```
export class AppComponent {  
  name = "John";  
  age = 22;  
}
```

■ Template

```
<h1>{{ name }}</h1>  
<p>Age: {{ age }}</p>  
<p>Status: {{ age > 18 ? "Adult" : "Minor" }}</p>
```

■ Limitations of Angular Expressions

1. No Loops or Conditions (like JS statements)

Cannot use:

```
if, for, while
```

2. No Assignments

Not allowed:

```
{{ x = 5 }}
```

3. No Access to Global Objects

Cannot use:

```
window, document
```

4. Only Simple Logic Allowed

- Complex logic should be in component

■ Advantages of Angular Expressions

1. Easy to Use
2. Dynamic Data Display
3. Automatic Updates
4. Cleaner Templates

■ Disadvantages

1. Limited Functionality

2. Cannot Handle Complex Logic
3. Overuse Can Affect Performance

■ Conclusion

Angular expressions are essential for:

- Displaying dynamic data
- Connecting UI with component logic

They provide a simple way to make applications interactive and responsive.

TIP: Exam Writing Tips

- Start with definition (expressions = dynamic values in templates)
- Write syntax: {{ }} (very important)
- Give examples (variable, condition, function)
- Mention limitations (important for marks)
- Use keywords: *interpolation, dynamic data, template*

Data Binding in Angular

■ Definition

- Data Binding in Angular is the process of connecting the component (TypeScript logic) with the view (HTML template) so that data can flow between them automatically.

■ Introduction

In web applications, we need to:

- Display data on UI
- Capture user input
- Update UI dynamically

Angular provides data binding mechanisms to synchronize data between:

- Component (Model)
- View (UI)

■ This reduces manual DOM manipulation and makes applications dynamic and interactive.

■ Types of Data Binding (Very Important)

Angular supports 4 types of data binding:

1. Interpolation (One-Way Binding)

■ Definition

Displays data from component → view

■ Syntax

```
{{ data }}
```

■ Example

```
name = "John";
```

```
<h1>{{ name }}</h1>
```

■ Output: John

■ Key Point

- Data flows one-way (Component → View)
2. Property Binding (One-Way Binding)

■ Definition

Binds component data to HTML element properties

■ Syntax

```
[elementProperty]="value"
```

■ Example

```
imageUrl = "logo.png";
```

```
<img [src]="imageUrl">
```

■ Sets image source dynamically

■ Key Point

- Used for attributes like `src`, `disabled`, etc.
3. Event Binding (One-Way Binding)

■ Definition

Handles user actions like clicks and events

■ Syntax

```
(event)="function()"
```


■ Example

```
clickMe() {  
  alert("Button Clicked");  
}
```

```
<button (click)="clickMe()">Click</button>
```

■ Key Point

- Data flows View → Component
4. Two-Way Data Binding (Very Important)

■ Definition

Synchronizes data between component ↔ view

■ Syntax

```
[(ngModel)]="data"
```

■ Example

```
name = "";
```

```
<input [(ngModel)]="name">  
<p>{{ name }}</p>
```

■ As user types → UI updates automatically

■ Key Point

- Combines:
- Property Binding + Event Binding

■ Example (All Data Binding Types Together)

■ Component

```
export class AppComponent {  
  name = "John";  
  imageUrl = "logo.png";  
  
  showMessage() {  
    alert("Hello");  
  }  
}
```

■ Template

```
<h1>{{ name }}</h1> <!-- Interpolation -->  
  
<img [src]="imageUrl"> <!-- Property Binding -->  
  
<button (click)="showMessage()">Click</button> <!-- Event Binding -->  
  
<input [(ngModel)]="name"> <!-- Two-Way Binding -->
```

■ Data Binding Flow

Component (Data) ■ View (UI)

■ Keeps UI updated automatically

■ Advantages of Data Binding

1. Reduces Code Complexity
2. Automatic UI Updates
3. Improves Development Speed
4. Better User Experience

■ Disadvantages

1. Performance Issues (if overused)
2. Complexity in Large Applications
3. Requires Understanding of Binding Types

■ Conclusion

Data binding is one of the most powerful features of Angular. It allows:

- Seamless communication between UI and logic
- Dynamic and interactive applications

Mastering data binding is essential for Angular development.

TIP: Exam Writing Tips

- Start with definition (binding = connection between component & view)

- Clearly write 4 types (very important)
- Include syntax for each type
- Add one combined example
- Highlight two-way binding (high scoring point)
- Use keywords: *interpolation, property binding, event binding, ngModel*

Built-in Directives in Angular

■ Definition

- Built-in Directives in Angular are predefined instructions provided by Angular that help in modifying the structure or appearance of DOM elements.

■ Introduction

In normal HTML, we cannot dynamically:

- Show/hide elements
- Repeat elements
- Change styles conditionally

Angular solves this using Directives.

■ Directives allow developers to:

- Control DOM behavior
- Add dynamic functionality to HTML

■ What are Directives?

A Directive is a class that adds behavior to elements in the DOM.

■ Angular provides:

- Built-in directives
- Custom directives

■ Types of Built-in Directives

Angular directives are mainly classified into:

1. Structural Directives (Very Important)

■ Definition

Structural directives change the structure of the DOM by adding or removing elements.

■ Key Feature

- Uses `*`` symbol

■ Common Structural Directives

■ (a) `*ngIf``

■ Purpose

- Displays element conditionally

■ Example

```
<div *ngIf="isLoggedIn">
  Welcome User
</div>
```

■ If `isLoggedIn = true` → shows

■ If false → hides

■ (b) `\*ngFor`

■ Purpose

- Repeats elements (loop)

■ Example

```
<ul>
  <li *ngFor="let item of items">
    {{ item }}
  </li>
</ul>
```

■ Displays list dynamically

■ (c) `\*ngSwitch`

■ Purpose

- Works like switch-case

■ Example

```
<div [ngSwitch]="role">
  <p *ngSwitchCase="'admin'">Admin</p>
  <p *ngSwitchCase="'user'">User</p>
  <p *ngSwitchDefault>Guest</p>
</div>
```

2. Attribute Directives

■ Definition

Attribute directives change the appearance or behavior of elements without changing structure.

■ Common Attribute Directives

■ (a) `ngClass`

■ Purpose

- Adds/removes CSS classes

■ Example

```
<p [ngClass]="{ 'highlight': isActive }">  
  Text  
</p>
```

■ (b) `ngStyle`

■ Purpose

- Applies inline styles dynamically

■ Example

```
<p [ngStyle]="{ color: 'red' }">  
  Styled Text  
</p>
```

■ (c) `ngModel` (Important)

■ Purpose

- Used for two-way data binding

■ Example

```
<input [(ngModel)]="name">
```

■ Example (Combined Directives)

■ Component

```
export class AppComponent {  
  isLoggedIn = true;  
  items = ["Angular", "React", "Vue"];  
}
```

■ Template

```
<!-- ngIf -->
<div *ngIf="isLoggedIn">Welcome</div>

<!-- ngFor -->
<ul>
  <li *ngFor="let item of items">{{ item }}</li>
</ul>

<!-- ngStyle -->
<p [ngStyle]="{ color: 'blue' }">Hello</p>
```

■ Advantages of Built-in Directives

1. Dynamic UI Control
2. Reduces Manual DOM Manipulation
3. Improves Code Readability
4. Powerful and Flexible

■ Disadvantages

1. Overuse Can Reduce Performance
2. Learning Curve for Beginners
3. Complex Templates in Large Apps

■ Conclusion

Built-in directives are essential in Angular for:

- Controlling UI dynamically
- Modifying DOM behavior
- Building interactive applications

They make Angular applications powerful and flexible.

TIP: Exam Writing Tips

- Start with definition (directives = DOM manipulation tools)
- Clearly explain 2 types: Structural & Attribute
- Include examples: ngIf, ngFor (very important)
- Mention ngModel (for extra marks)
- Use keywords: *DOM, structural, attribute, ngIf, ngFor*

Custom Directives in Angular

■ Definition

- Custom Directives in Angular are user-defined directives that allow developers to create their own DOM behavior and reusable functionality beyond the built-in directives.

■ Introduction

Angular provides many built-in directives like:

- `\*ngIf`
- `\*ngFor`

However, sometimes applications require custom behavior such as:

- Changing color dynamically
- Highlighting elements
- Adding custom effects

■ In such cases, developers create Custom Directives.

■ Why Custom Directives are Needed

1. Extend HTML Functionality
 - Add new behavior to elements
2. Code Reusability
 - Write once, use multiple times
3. Cleaner Code
 - Avoid repeated logic

■ Types of Custom Directives

1. Attribute Directives (Most Common)
 - Modify appearance or behavior
2. Structural Directives
 - Modify DOM structure (advanced usage)

■ Steps to Create Custom Directive

■ Step 1: Generate Directive

```
ng generate directive highlight
```

■ Creates:

```
highlight.directive.ts
```

■ Step 2: Basic Directive Structure

```
import { Directive } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  constructor() { }
}
```

■ Example (Custom Attribute Directive)

■ Goal: Highlight text in yellow color

■ Directive Code

```
import { Directive, ElementRef } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {

  constructor(private el: ElementRef) {
    this.el.nativeElement.style.backgroundColor = 'yellow';
  }

}
```

■ Usage in HTML

```
<p appHighlight>
  This text is highlighted
</p>
```

■ Output: Text with yellow background

■ Example with Event Handling (Advanced)

■ Directive Code

```
import { Directive, ElementRef, HostListener } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {

  constructor(private el: ElementRef) {}

  @HostListener('mouseenter') onMouseEnter() {
    this.el.nativeElement.style.backgroundColor = 'yellow';
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.el.nativeElement.style.backgroundColor = 'white';
  }

}
```


■ Usage

```
<p appHighlight>
  Hover over me
</p>
```

■ Changes color on hover

■ Key Concepts Used

1. `@Directive`
 - Decorator to define directive
2. `selector`
 - Used as HTML attribute
3. `ElementRef`
 - Access DOM element
4. `HostListener`
 - Listen to events

■ Advantages of Custom Directives

1. Reusability
2. Cleaner UI Logic
3. Extend Angular Functionality
4. Improves Maintainability

■ Disadvantages

1. Direct DOM Manipulation Risks
2. Complex for Beginners
3. Debugging Difficulty

■ Conclusion

Custom directives allow developers to:

- Extend Angular functionality
- Create reusable UI behaviors
- Improve application design

They are essential for building custom and dynamic user interfaces.

TIP: Exam Writing Tips

- Start with definition (custom directive = user-defined directive)
- Explain why needed
- Mention types (attribute, structural)
- Include example using ElementRef (important)
- Add HostListener example for extra marks
- Use keywords: *directive, selector, ElementRef, DOM*

Implementing Angular Services

■ Definition

An Angular Service is a reusable class that contains business logic, data handling, or shared functionality, which can be used across multiple components using Dependency Injection (DI).

■ Introduction

In Angular applications, components should mainly handle:

- UI (view)
- User interaction

But business logic like:

- API calls
- Data processing
- Shared data

should be placed in Services.

■ This separation improves:

- Code organization
- Reusability
- Maintainability

■ Why Services are Needed

1. Code Reusability
 - Same logic used in multiple components
2. Separation of Concerns
 - UI and logic are separated
3. Data Sharing
 - Share data between components
4. API Handling
 - Manage HTTP requests

■ Steps to Create and Use Service

■ Step 1: Generate Service

```
ng generate service data
```

■ Creates:

```
data.service.ts
```

■ Step 2: Service Code

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class DataService {

  constructor() { }

  getStudents() {
    return ["John", "Mike", "Ali"];
  }

}
```

■ Explanation

- `@Injectable()` → Marks class as service
- `providedIn: 'root'` → Available globally

■ Step 3: Use Service in Component

■ Component Code

```
import { Component } from '@angular/core';
import { DataService } from '../data.service';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {

  students: string[] = [];

  constructor(private dataService: DataService) {
    this.students = this.dataService.getStudents();
  }

}
```

■ Template Code

```
<ul>
  <li *ngFor="let s of students">{{ s }}</li>
</ul>
```

■ Output:

- John
- Mike
- Ali

■ Role of Dependency Injection (Very Important)

■ Definition

Dependency Injection is a mechanism where Angular automatically provides required services to components.

■ Example

```
constructor(private dataService: DataService) {}
```

■ No need to manually create object

■ Real-World Example

■ E-commerce Application

- ProductService → fetch products
- UserService → manage users
- OrderService → handle orders

■ Advantages of Angular Services

1. Reusability
2. Cleaner Code Structure
3. Easy Data Sharing
4. Centralized Logic

■ Disadvantages

1. Overuse Can Increase Complexity
2. Requires DI Understanding
3. Debugging Can Be Difficult

■ Conclusion

Angular services are essential for:

- Managing business logic
- Sharing data between components
- Building scalable applications

Using Dependency Injection, Angular makes services easy to use and manage.

TIP: Exam Writing Tips

- Start with definition (service = reusable logic class)
- Explain why services are needed
- Include steps: generate → create → use
- Highlight Dependency Injection (very important)

- Add example with component usage
- Use keywords: *service, DI, reusable, business logic*

UNIT-IV COMPLETED
